



University of Asia Pacific

Department of Computer Science and Engineering

CSE 316: Microprocessors and

Microcontrollers Lab

PROJECT REPORT

CODE FOR GRAB AND GO

Submitted by:

Name : Md Arkive , Md. Zobaer Islam , Siddatul Muntaha Chowdhury Mahin

Student ID : 22201149,22201151,22201158

Section : C2

Submitted to:

Zaima Sartaj Taheri

Lecturer,

Department of Computer Science and Engineering

CODE For Arm and Car with Control :

```
#include <ESP8266WiFi.h>

#include <ESP8266WebServer.h>

#include <Servo.h>

// ----- Motor Pins (adjust as needed for your setup) -----

const int ENA = D1;    // PWM pin for left motor enable

const int IN1 = D2;

const int IN2 = D3;

const int IN3 = D5;

const int IN4 = D6;

const int ENB = D7;    // PWM pin for right motor enable


const int motorSpeed = 800;  // 0-1023 PWM speed


// ----- Servo Pins -----

const int servoPins[] = {D0, D8, A0, D4, D9, D10}; // You can adjust if
needed

Servo servos[6];


// ----- Create Web Server on port 80 -----

ESP8266WebServer server(80);
```

```
// ----- HTML + CSS + JS -----

const char* htmlPage = R"rawliteral(

<!DOCTYPE html>

<html>

<head>

    <title>Wemos D1 Robot Control</title>

    <style>

        body { text-align: center; font-family: Arial; }

        button, input[type=range] { margin: 10px; padding: 10px 30px;
font-size: 18px; }

        .servo { margin-bottom: 20px; }

    </style>

    <script>

        function sendCmd(cmd) {

            fetch("/") + cmd);

        }

        function setServo(id, val) {

            fetch(/servo?num=${id}&angle=${val});

            document.getElementById("val" + id).innerText = val;

        }

    </script>

</head>

<body>

    <h2>Wemos D1 Robot & Servo Control</h2>

    <button onclick="sendCmd('forward') ">FORWARD</button><br>
```

```

<button onclick="sendCmd('left')">LEFT</button>

<button onclick="sendCmd('right')">RIGHT</button><br>

<button onclick="sendCmd('backward')">BACKWARD</button><br>

<button onclick="sendCmd('stop')" style="background:red;
color:white;">STOP</button><br><br>

<h3>Servo Control</h3>

%SERVO_CONTROLS%

</body>

</html>

)rawliteral";

// ----- Dynamic Servo Sliders -----

String getServoControls() {

    String html = "";

    for (int i = 0; i < 6; i++) {

        int minAngle = (i == 4) ? 0 : 0;

        int maxAngle = (i == 4) ? 20 : 180;

        int defaultAngle = (i == 4) ? 0 : 90;

        html += "<div class='servo'>Servo " + String(i) +

            ": <span id='val" + String(i) + "'>" + String(defaultAngle) +

            "</span><br>" +

            "<input type='range' min='" + String(minAngle) + "' max='" +

            String(maxAngle) +

            "' value='" + String(defaultAngle) +

```

```

        "' oninput='setServo(" + String(i) + ", this.value)'></div>";

    }

    return html;
}

// ----- Motor Control Functions -----

void moveForward() {

    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);

    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);

    analogWrite(ENA, motorSpeed);

    analogWrite(ENB, motorSpeed);

    server.send(200, "text/plain", "Forward");

}

void moveBackward() {

    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);

    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);

    analogWrite(ENA, motorSpeed);

    analogWrite(ENB, motorSpeed);

    server.send(200, "text/plain", "Backward");

}

void turnLeft() {

    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);

    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);

```

```
    analogWrite(ENA, motorSpeed);

    analogWrite(ENB, motorSpeed);

    server.send(200, "text/plain", "Left");
}

void turnRight() {

    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);

    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);

    analogWrite(ENA, motorSpeed);

    analogWrite(ENB, motorSpeed);

    server.send(200, "text/plain", "Right");
}

void stopMotors() {

    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);

    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);

    analogWrite(ENA, 0);

    analogWrite(ENB, 0);

    server.send(200, "text/plain", "Stopped");
}

// ----- Handle Web Root -----

void handleRoot() {

    String page = htmlPage;

    page.replace("%SERVO_CONTROLS%", getServoControls());
```

```

    server.send(200, "text/html", page);
}

// ----- Servo Control Handler -----

void handleServo() {

    if (server.hasArg("num") && server.hasArg("angle")) {

        int num = server.arg("num").toInt();

        int angle = server.arg("angle").toInt();

        if (num >= 0 && num < 6) {

            if (num == 4) angle = constrain(angle, 0, 20);

            else angle = constrain(angle, 0, 180);

            servos[num].write(angle);

            server.send(200, "text/plain", "Servo updated");

        } else {

            server.send(400, "text/plain", "Invalid servo");

        }

    } else {

        server.send(400, "text/plain", "Missing parameters");

    }

}

void setup() {

    Serial.begin(115200);

    // Motor pins

```

```
pinMode(ENA, OUTPUT);

pinMode(IN1, OUTPUT);

pinMode(IN2, OUTPUT);

pinMode(IN3, OUTPUT);

pinMode(IN4, OUTPUT);

pinMode(ENB, OUTPUT);


// Servo setup
for (int i = 0; i < 6; i++) {

    servos[i].attach(servoPins[i]);

    if (i == 4) servos[i].write(0);

    else servos[i].write(90);

}


// Wi-Fi AP Mode

WiFi.softAP("WEMOS_ROBOT", "12345678");

Serial.println("Access Point: WEMOS_ROBOT");

Serial.println("Visit: http://192.168.4.1");


// Routes

server.on("/", handleRoot);

server.on("/forward", moveForward);

server.on("/backward", moveBackward);

server.on("/left", turnLeft);

server.on("/right", turnRight);
```



```

server.on("/stop", stopMotors);

server.on("/servo", handleServo);

server.begin();

Serial.println("Server started.");
}

void loop() {
    server.handleClient();
}

```

CODE For CAM :

```

#include "esp_camera.h"

#include <WiFi.h>

//

// WARNING!!! PSRAM IC required for UXGA resolution and high JPEG quality
//           Ensure ESP32 Wrover Module or other board with PSRAM is
//           selected

//           Partial images will be transmitted if image exceeds buffer
//           size

//

//           You must select partition scheme from the board menu that
//           has at least 3MB APP space.

//           Face Recognition is DISABLED for ESP32 and ESP32-S2, because
//           it takes up from 15

//           seconds to process single frame. Face Detection is ENABLED
//           if PSRAM is enabled as well

```

```

// =====

// Select camera model

// =====

//#define CAMERA_MODEL_WROVER_KIT // Has PSRAM

//#define CAMERA_MODEL_ESP_EYE // Has PSRAM

//#define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM

//#define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM

//#define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has PSRAM

//#define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM

//#define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM

//#define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM

#define CAMERA_MODEL_AI_THINKER // Has PSRAM

//#define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM

// ** Espressif Internal Boards **

//#define CAMERA_MODEL_ESP32_CAM_BOARD

//#define CAMERA_MODEL_ESP32S2_CAM_BOARD

//#define CAMERA_MODEL_ESP32S3_CAM_LCD

#include "camera_pins.h"

// =====

// Enter your WiFi credentials

// =====

const char* ssid = "Nazmul";

const char* password = "987654321";

void startCameraServer();

```

```
void setup() {  
  
    Serial.begin(115200);  
  
    Serial.setDebugOutput(true);  
  
    Serial.println();  
  
  
    camera_config_t config;  
  
    config.ledc_channel = LEDC_CHANNEL_0;  
    config.ledc_timer = LEDC_TIMER_0;  
    config.pin_d0 = Y2_GPIO_NUM;  
    config.pin_d1 = Y3_GPIO_NUM;  
    config.pin_d2 = Y4_GPIO_NUM;  
    config.pin_d3 = Y5_GPIO_NUM;  
    config.pin_d4 = Y6_GPIO_NUM;  
    config.pin_d5 = Y7_GPIO_NUM;  
    config.pin_d6 = Y8_GPIO_NUM;  
    config.pin_d7 = Y9_GPIO_NUM;  
    config.pin_xclk = XCLK_GPIO_NUM;  
    config.pin_pclk = PCLK_GPIO_NUM;  
    config.pin_vsync = VSYNC_GPIO_NUM;  
    config.pin_href = HREF_GPIO_NUM;  
    config.pin_sscb_sda = SIOD_GPIO_NUM;  
    config.pin_sscb_scl = SIOC_GPIO_NUM;  
    config.pin_pwdn = PWDN_GPIO_NUM;  
    config.pin_reset = RESET_GPIO_NUM;  
    config.xclk_freq_hz = 20000000;  
    config.frame_size = FRAMESIZE_UXGA;
```

```

    config.pixel_format = PIXFORMAT_JPEG; // for streaming

    //config.pixel_format = PIXFORMAT_RGB565; // for face
detection/recognition

    config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;

    config.fb_location = CAMERA_FB_IN_PSRAM;

    config.jpeg_quality = 12;

    config.fb_count = 1;


    // if PSRAM IC present, init with UXGA resolution and higher JPEG
quality
    //                               for larger pre-allocated frame buffer.
    if(config.pixel_format == PIXFORMAT_JPEG){
        if(psramFound()){
            config.jpeg_quality = 10;

            config.fb_count = 2;

            config.grab_mode = CAMERA_GRAB_LATEST;

        } else {

            // Limit the frame size when PSRAM is not available

            config.frame_size = FRAMESIZE_SVGA;

            config.fb_location = CAMERA_FB_IN_DRAM;

        }

    } else {

        // Best option for face detection/recognition

        config.frame_size = FRAMESIZE_240X240;

#ifdef CONFIG_IDF_TARGET_ESP32S3
        config.fb_count = 2;
#endif

    }

```

```

#ifdef CAMERA_MODEL_ESP_EYE

    pinMode(13, INPUT_PULLUP);

    pinMode(14, INPUT_PULLUP);

#endif

// camera init
esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {

    Serial.printf("Camera init failed with error 0x%x", err);

    return;

}

sensor_t * s = esp_camera_sensor_get();

// initial sensors are flipped vertically and colors are a bit saturated
if (s->id.PID == OV3660_PID) {

    s->set_vflip(s, 1); // flip it back

    s->set_brightness(s, 1); // up the brightness just a bit

    s->set_saturation(s, -2); // lower the saturation

}

// drop down frame size for higher initial frame rate
if(config.pixel_format == PIXFORMAT_JPEG){

    s->set_framesize(s, FRAMESIZE_QVGA);

}

#ifdef CAMERA_MODEL_M5STACK_WIDE ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)

    s->set_vflip(s, 1);

```

```
s->set_hmirror(s, 1);
#endif

#ifdef CAMERA_MODEL_ESP32S3_EYE
s->set_vflip(s, 1);
#endif

WiFi.begin(ssid, password);
WiFi.setSleep(false);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("");
Serial.println("WiFi connected");

startCameraServer();

Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
Serial.println("' to connect");
}

void loop() {
    // Do nothing. Everything is done in another task by the web server
    delay(10000);
}
```

